

CAP6411

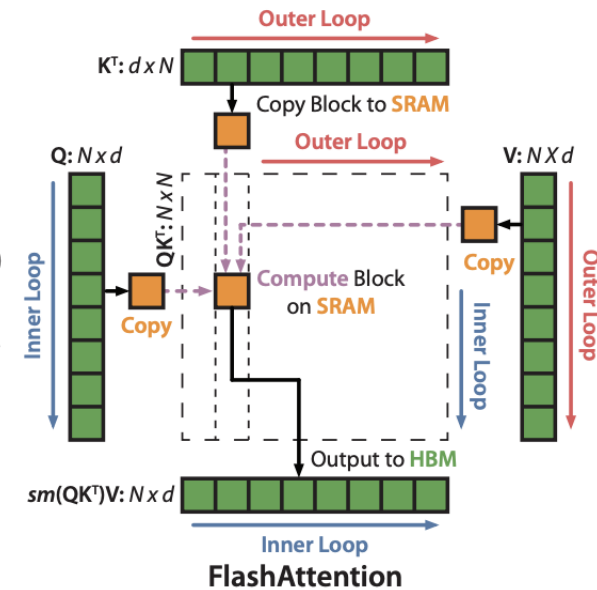
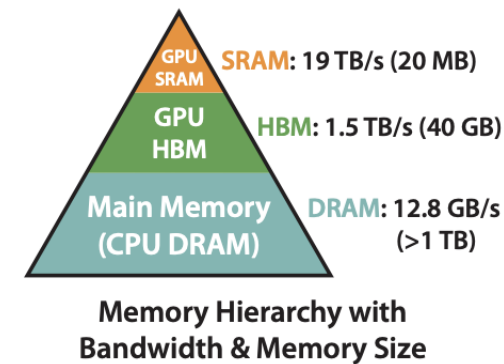
Model Efficiency via Memory "Caching"

FlashAttention

- Specifically tailored for self-attention

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^T \in \mathbb{R}^{N \times N}, \quad \mathbf{P} = \text{softmax}(\mathbf{S}) \in \mathbb{R}^{N \times N}, \quad \mathbf{O} = \mathbf{P}\mathbf{V} \in \mathbb{R}^{N \times d}$$

- Idea is to break the QKV and input into small blocks and minimize swopping between SRAM (fastest) and HBM ram (slower but larger cap)
- Goal is to minimize the swopping between SRAM and HBM
- Exact solution by maintaining running matrices
- Reading assignment**

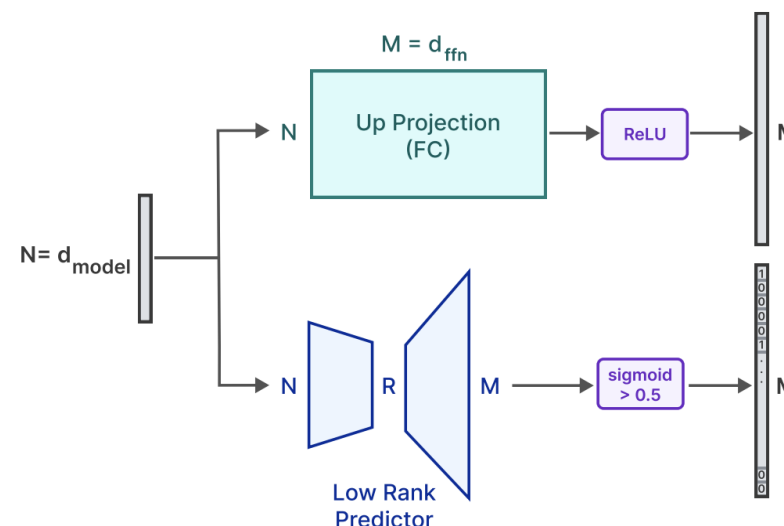


LLM in a Flash

- Based off older work
 - When GPU memory is DRAM and flash (slower but larger)
 - Of course there is CPU memory
- But has interesting ideas
 - How to reduce the number of data transfer between DRAM and flash
 - Increasing transfer throughput
 - Optimized data management in DRAM

Principle 1: Reduce Number of Data Transfer

- Note: retain all of the attention embeddings and matrices in the DRAM (as opposed to flashattention)
- Load only non-sparse segments into DRAM (~1/3 of an OPT model)
- Anticipate ReLU sparsity
 - Low Rank predictor
- Sliding Window Strategy
 - Active neuron – predicted by low rank predictor
 - Delete and add strategy



Zero-Shot Task	OPT 6.7B	with Predictor
Arc Easy	66.1	66.2
Arc Challenge	30.6	30.6
HellaSwag	50.3	49.8

Principle 2: Increasing Transfer Throughput

- Larger chunks per transfer!!
 - In FFN, lump the rows and columns together
 - Bundling co-activation neurons did not work
- My research supports this as well

Principle 3: Optimize Data Management in DRAM

- While DRAM is the fastest
- Moving chunks in and out still incur cost
 - Deletion of old chunks
 - Addition of new chunks
- Make use of memory pointer
 - Don't delete, move pointer
 - Add when necessary
 - C programming
 - CudaAlloc(...)

Summary of Learning to be Efficient

- Distillation
 - Exercise: SDXL Turbo
- Mixture of "smaller" experts
 - Mixtral
- Memory caching

CAP6411

ResNet

Residual Network (ResNet)

- Before ResNet:
 - Adding more layer to deep CNN results in degradation of performance
 - Is this degradation due to overfitting?
 - No ...
 - Because training error went up – if it has been overfitting, training error would have gone down
 - If not overfitting, should adding more layers cause degradation by itself?
 - No ...
 - Imagine a shallow network, you can add more layers that are identity mappings, so the resulting deeper network should minimally have the same performance!!

What does this tell us?

- Trying to learn even an identity mapping as network gets deeper is not trivial
- So, what's the solution?
 - Say $\mathcal{H}(\mathbf{x})$ is the desired mapping
 - We learn $\mathcal{F}(\mathbf{x}) := \mathcal{H}(\mathbf{x}) - \mathbf{x}$ instead
 - The original desired mapping is now $\mathcal{F}(\mathbf{x}) + \mathbf{x}$
 - This means that if we want to learn identity mappings, we just need to learn $\mathbf{x} = 0$, which is easier than learning an identity mapping as mentioned

Residual Block

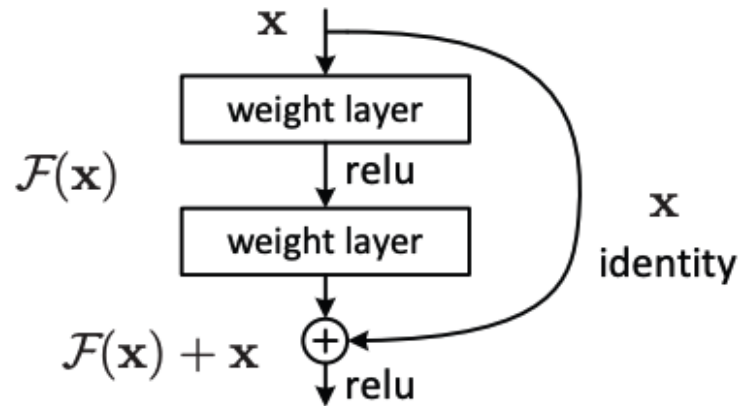


Figure 2. Residual learning: a building block.

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}.$$

$$\mathcal{F} = W_2 \sigma(W_1 \mathbf{x})$$

- Shortcut connections
- Element-wise addition
- 2-3 layers work best per block

Implementations

3.4. Implementation

Our implementation for ImageNet follows the practice in [21, 41]. The image is resized with its shorter side randomly sampled in $[256, 480]$ for scale augmentation [41]. A 224×224 crop is randomly sampled from an image or its horizontal flip, with the per-pixel mean subtracted [21]. The standard color augmentation in [21] is used. We adopt batch normalization (BN) [16] right after each convolution and before activation, following [16]. We initialize the weights as in [13] and train all plain/residual nets from scratch. We use SGD with a mini-batch size of 256. The learning rate starts from 0.1 and is divided by 10 when the error plateaus, and the models are trained for up to 60×10^4 iterations. We use a weight decay of 0.0001 and a momentum of 0.9. We do not use dropout [14], following the practice in [16].

In testing, for comparison studies we adopt the standard 10-crop testing [21]. For best results, we adopt the fully-convolutional form as in [41, 13], and average the scores at multiple scales (images are resized such that the shorter side is in $\{224, 256, 384, 480, 640\}$).

Assignments (due 9/10)

- Train on Human Action Recognition (40%) <https://www.kaggle.com/datasets/shashankrapolu/human-action-recognition-dataset/data>
 - Evaluation on test set human-action-recognition
 - 15 classes, 218 MB, 12600 images, train/test split available
 - Use Newton
 - ResNet 18
- Report and code zip (10%)
 - Errors and obstacles faced running the model
 - Must have conda requirement.txt, cli commands to generate the evaluation results above (random checks will be performed to verify)
 - Training log
- Video of live demo (20%)
 - The video should comprise of visual outputs (e.g., a text "cat" overlay on the image being classified that has a cat in it)
- Insights on speed and memory efficiency (30%)
 - Bonus 100 points: if you can apply what we have discussed on model efficiency (distillation, SMoE, "caching") with demonstrated > 1.5x speed up, before and after